

CPS 3440/5014: Analysis of Algorithms

Dr. Yan Ma

yama@kean.edu

yan.ma@kean.edu

What in This Course?

The theoretical study of design and analysis of computer algorithms

Basic goals for an algorithm:

- always correct
- always terminates
- This class: performance
 - Performance often draws the line between what is possible and what is impossible.

Design and Analysis of Algorithms

- *Analysis:* predict the cost of an algorithm in terms of resources and performance
- *Design:* design algorithms which minimize the cost

The Challenge

- *Why is my program so slow?*
 - ➡ Running-time efficiency
- *Why does it run out of memory?*
 - ➡ Space efficiency

Review: Induction

- *Suppose*
 - $S(k)$ is true for fixed constant k
 - $S(n) \Rightarrow S(n + 1)$ for all $n \geq k$
- *Then*
 - $S(n)$ is true for all $n \geq k$

Review: Proof by Induction

- **Claim:** $S(n)$ is true for all $n \geq k$
- ***Base case $S(k)$:***
 - Show the statement (formula) is true for the initial value $n = k$
- ***Induction Hypothesis:***
 - Assume it's true for an arbitrary n , $n \geq k$
- ***Induction Step:***
 - Show that if the statement holds for n , then it also holds for $n + 1$.

Induction Example #1

Q: Prove $1 + 2 + 3 + \dots + n = n(n+1) / 2$, $n \geq 1$

A:

1. **Base case:** If $n = 1$, then $1 = 1(1+1) / 2$

2. **Inductive hypothesis:**

Assume $1 + 2 + 3 + \dots + n = n(n+1) / 2$

3. **Inductive step** (show true for $n+1$):

$$\begin{aligned} 1 + 2 + \dots + n + n+1 &= (1 + 2 + \dots + n) + (n+1) \\ &= n(n+1)/2 + n+1 = [n(n+1) + 2(n+1)]/2 \\ &= (n+1)(n+2)/2 = (n+1)(n+1 + 1) / 2 \end{aligned}$$

Induction Example #2

Q: Prove $2^n \geq n + 1$, $n \geq 0$

A:

1. **Base case:** If $n = 0$, then $2^0 = 1 \geq 0 + 1 = 1$

2. **Inductive hypothesis:**

Assume $2^n \geq n + 1$ for some arbitrary $n \geq 0$

3. **Inductive step** (show true for $n+1$):

$$2^{n+1} = 2 \cdot 2^n \geq 2(n + 1)$$

$$= n + 1 + 1 + n \geq (n + 1) + 1, \text{ if } n \geq 0$$

Our Machine Model

Generic Random Access Machine (RAM)

- Executes operations sequentially
- Set of primitive operations:
 - Arithmetic. Logical, Comparisons, Function calls
- Simplifying assumption: all ops cost 1 unit
 - Eliminates dependence on the speed of our computer, otherwise impossible to verify and to compare

Input Size

- **Time and space complexity**
 - This is generally **a function of the input size**
 - E.g., sorting, multiplication
 - **How we measure input size depends:**
 - Sorting: number of input items
 - Multiplication: total number of bits
 - Graph algorithms: number of nodes & edges
 - Etc

Running Time

- **Number of primitive steps that are executed**
 - Except for time of executing a function call most statements roughly require the same amount of time

Kinds of Analyses

Worst-case: (usually)

- Provides **an upper bound on running time**
- An absolute guarantee
- Somewhat pessimistic: one bad input → ”bad” algorithm.

Average-case: (sometimes)

- Provides the **expected running time**
- Very useful, but treat with care: what is “average”?
 - Random (equally likely) inputs
 - Real-life inputs

HARD TO DO !

Best-case: (NEVER)

- Cheat with a slow algorithm that works fast on *some* input.

Asymptotic Performance

- In this course, we care most about *asymptotic performance*
 - How does the algorithm behave as the problem size gets very large?
 - Running time
 - Memory/storage requirements
 - Bandwidth/power requirements/logic gates/etc.

Example: Sorting Problem

Input: sequence $\langle a_1, a_2, \dots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Example:

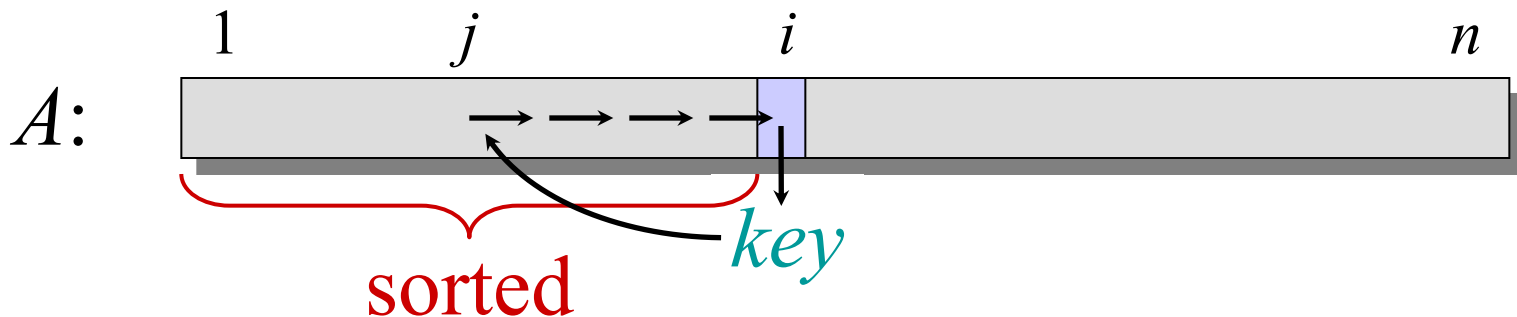
Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Insertion Sort

“pseudocode”

```
INSERTION-SORT ( $A, n$ )    ▷  $A[1 \dots n]$   
  for  $i \leftarrow 2$  to  $n$   
    do  $key \leftarrow A[i]$   
       $j \leftarrow i - 1$   
      while  $j > 0$  and  $A[j] > key$   
        do  $A[j+1] \leftarrow A[j]$   
           $j \leftarrow j - 1$   
       $A[j+1] = key$ 
```



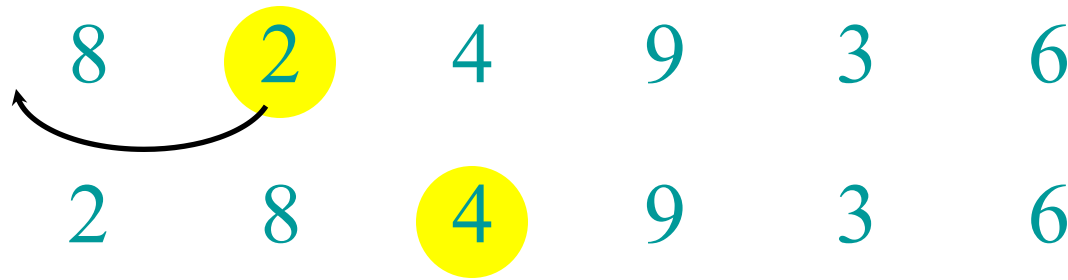
Example of Insertion Sort

8 2 4 9 3 6

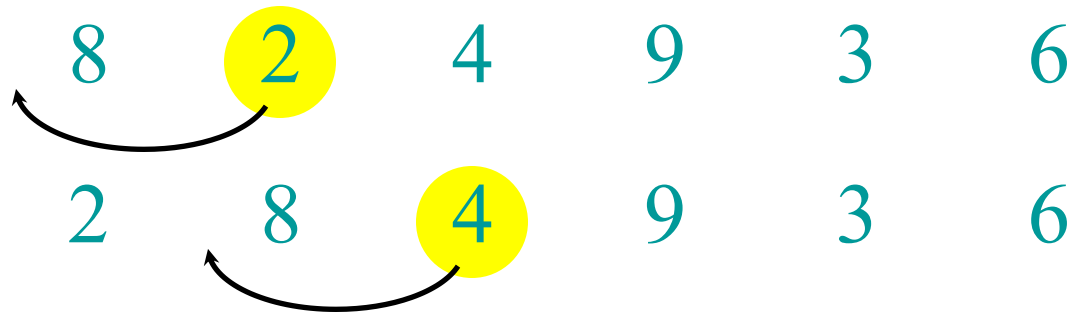
Example of Insertion Sort



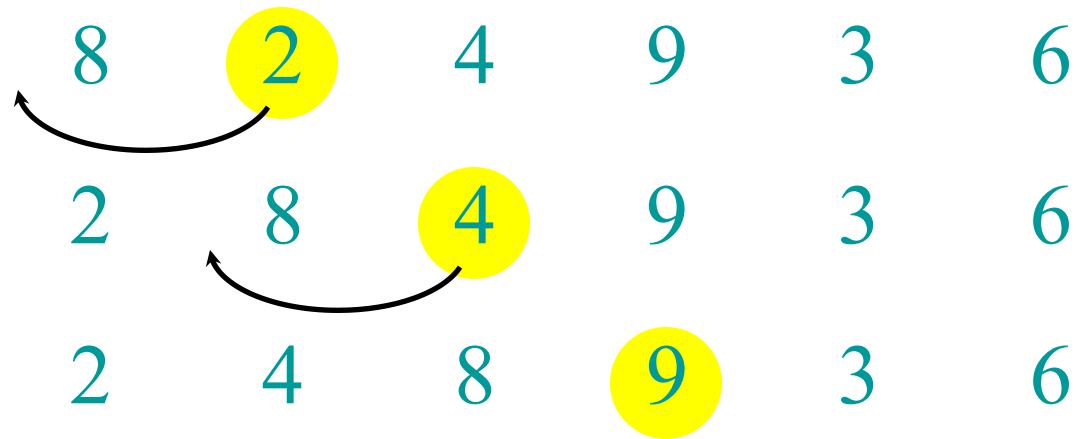
Example of Insertion Sort



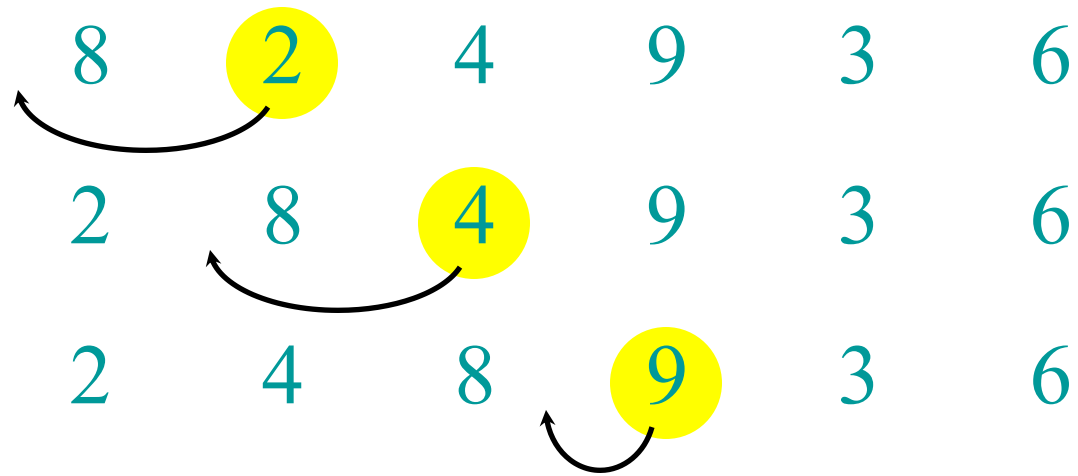
Example of Insertion Sort



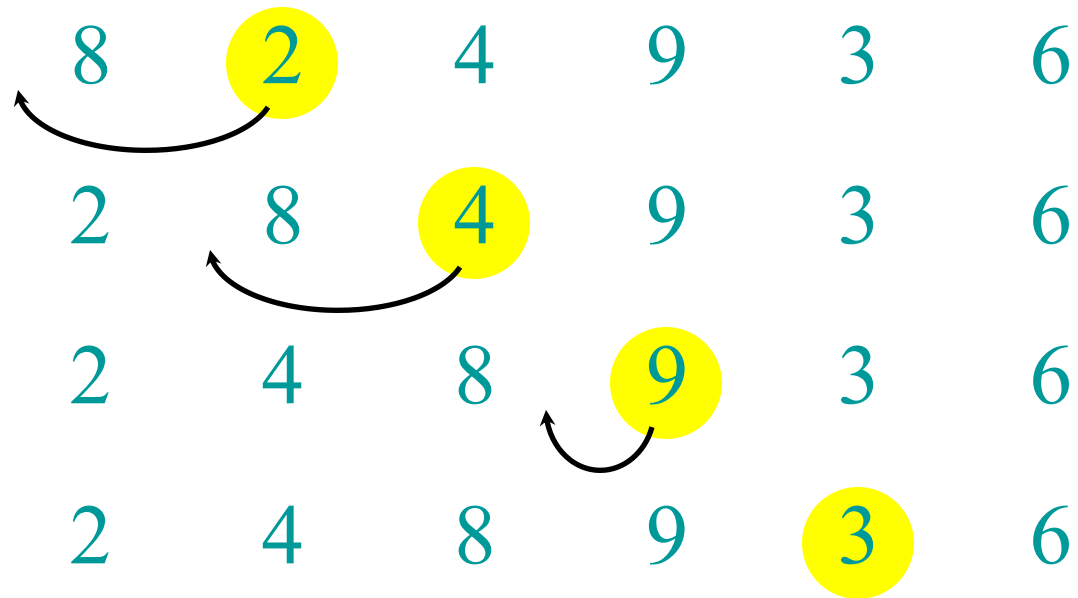
Example of Insertion Sort



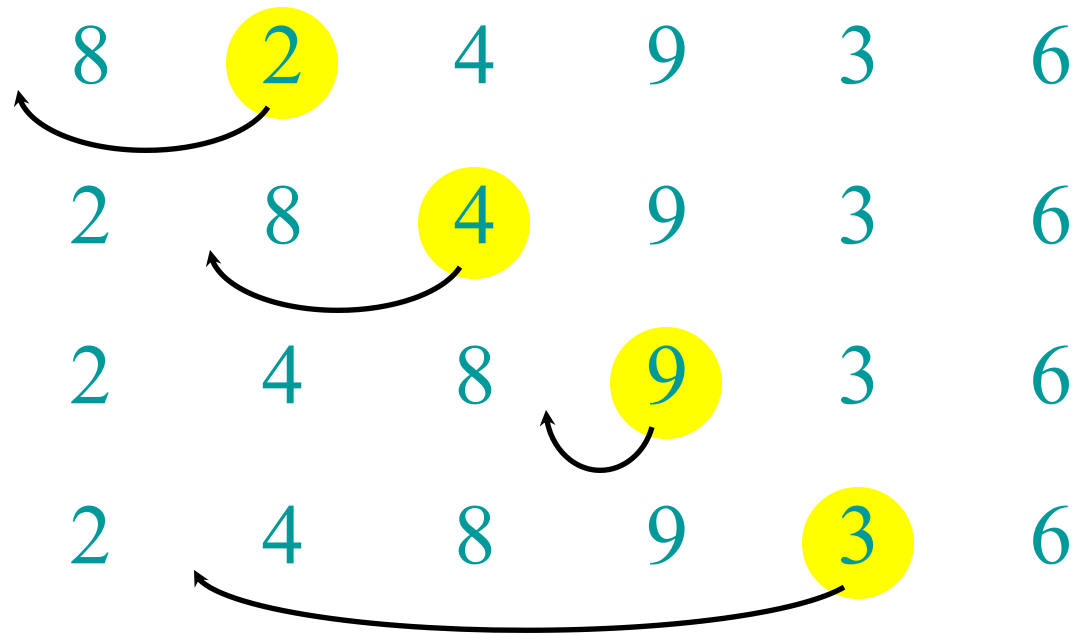
Example of Insertion Sort



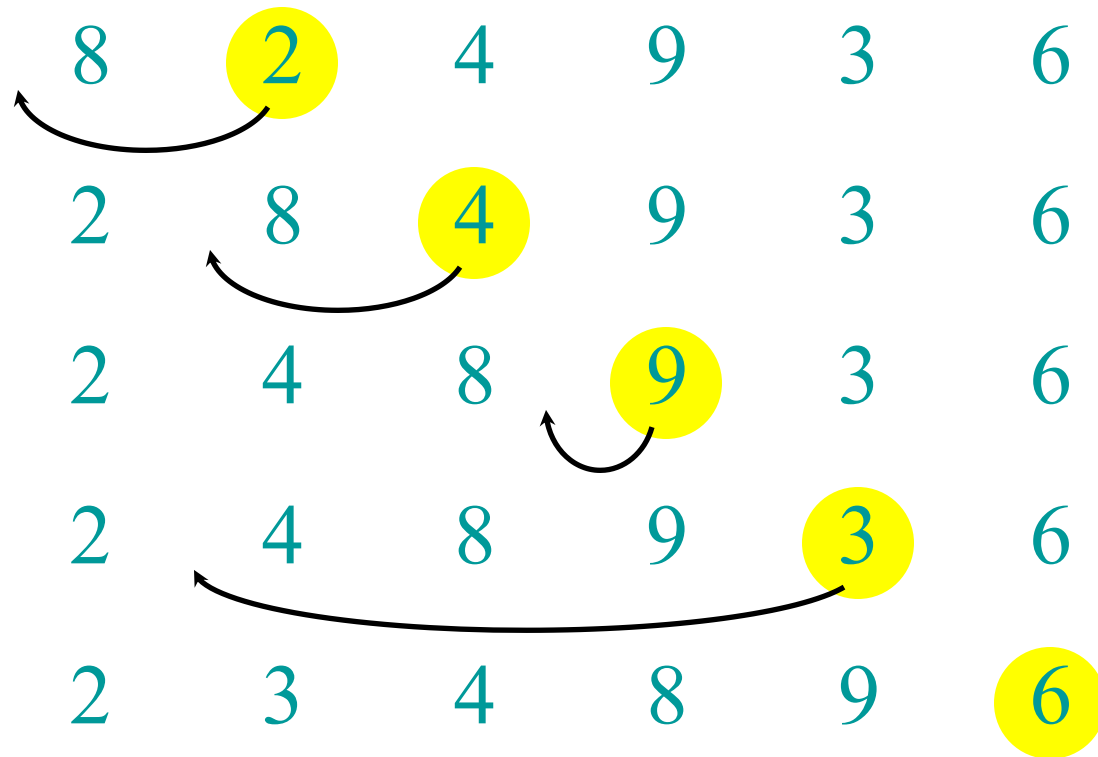
Example of Insertion Sort



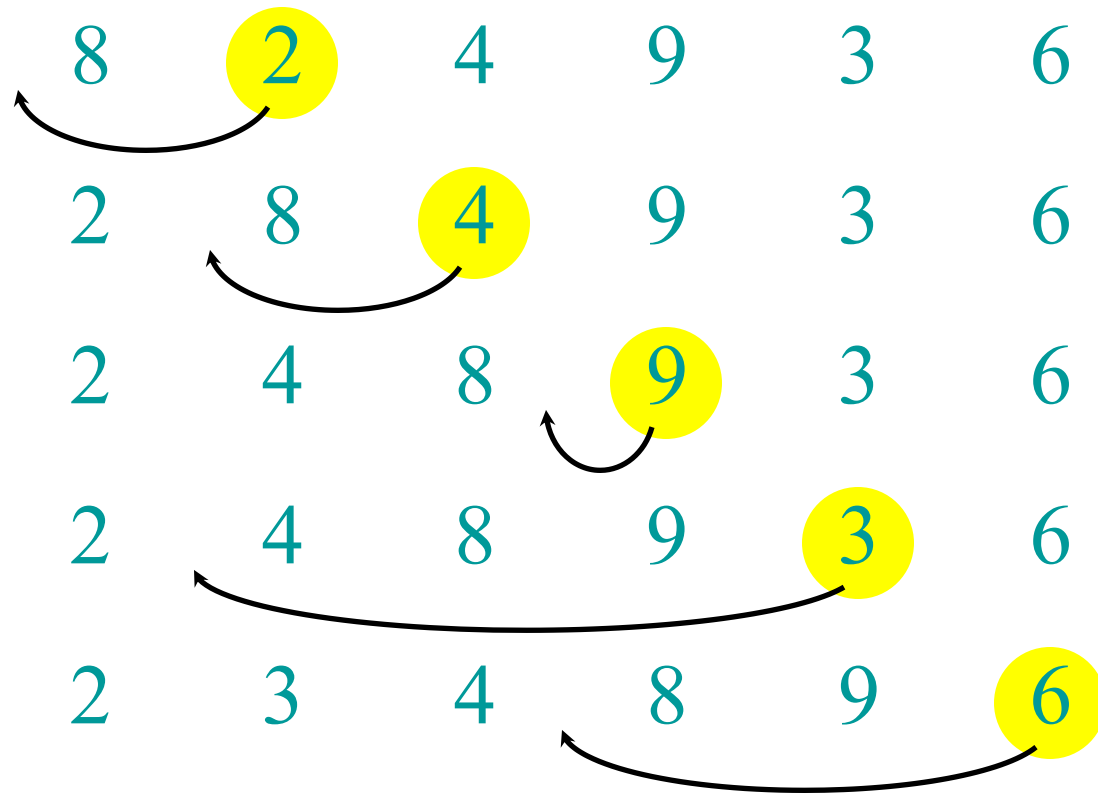
Example of Insertion Sort



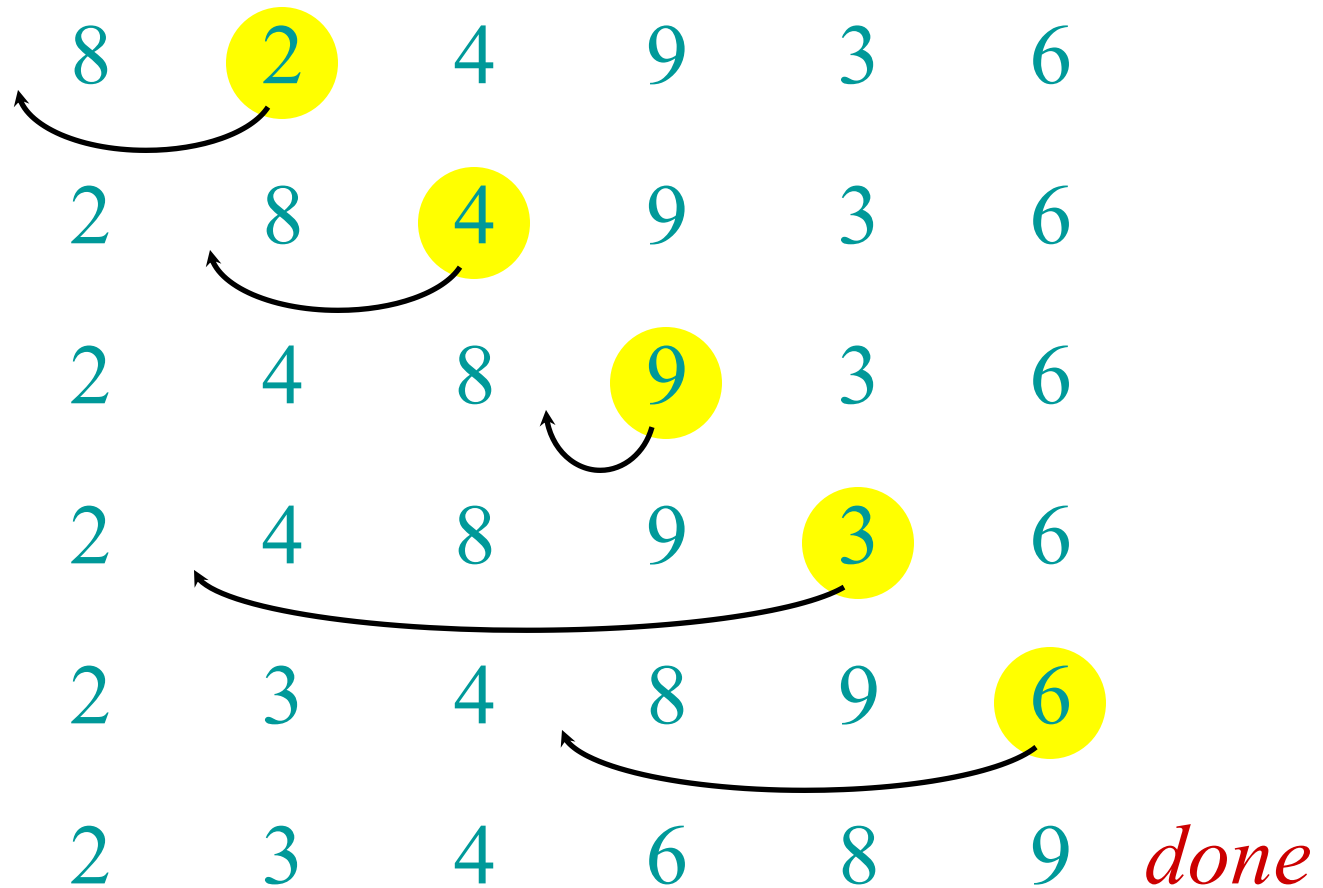
Example of Insertion Sort



Example of Insertion Sort



Example of Insertion Sort



Running Time

- The running time depends on the input: an already sorted sequence is easier to sort.
- **Major Simplifying Convention:**
Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
 - $T_A(n)$ = time of A on length n inputs
- Generally, we seek upper bounds on the running time, to have a guarantee of performance.

Kinds of Analyses

Worst-case: (usually)

- $T(n)$ = maximum time of algorithm on any input of size n .

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n .
- Need assumption of statistical distribution of inputs.

Machine-independent Time

What is insertion sort's worst-case time?

Depends on the speed of our computer: relative vs.
absolute speed

BIG IDEAS:

- *Ignore machine dependent constants,*
otherwise impossible to verify and to compare algorithms
- Look at *growth* of $T(n)$ as $n \rightarrow \infty$.

“Asymptotic Analysis”

Insertion Sort

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = \emptyset$	$j = \emptyset$	$key = \emptyset$
$A[j] = \emptyset$	$A[j+1] = \emptyset$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 10$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 30$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 1$	$\text{key} = 10$
$A[j] = 30$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 2$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 10$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 10$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 40$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 0$	$\text{key} = 40$
$A[j] = \emptyset$	$A[j+1] = 10$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 40$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$key = 20$
$A[j] = 30$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 40$	

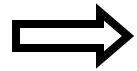


```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 3$	$\text{key} = 20$
$A[j] = 40$	$A[j+1] = 20$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$	$j = 3$	$\text{key} = 20$
$A[j] = 40$	$A[j+1] = 20$	

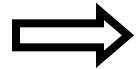
```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$key = 20$
$A[j] = 40$	$A[j+1] = 40$	

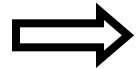


```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$key = 20$
$A[j] = 40$	$A[j+1] = 40$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 3$	$\text{key} = 20$
$A[j] = 40$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 40$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 40$	

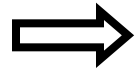
➔

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 30$	



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 2$	$\text{key} = 20$
$A[j] = 30$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 1$	$\text{key} = 20$
$A[j] = 10$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$	$j = 1$	$\text{key} = 20$
$A[j] = 10$	$A[j+1] = 30$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	20	30	40
1	2	3	4

$i = 4$	$j = 1$	$key = 20$
$A[j] = 10$	$A[j+1] = 20$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	20	30	40
1	2	3	4

$i = 4$	$j = 1$	$key = 20$
$A[j] = 10$	$A[j+1] = 20$	

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

Done!

Animating Insertion Sort

- Check out the Animator, a java applet at:
[Insertion Sort visualize | Algorithms | HackerEarth](#)
- Try it out with random, ascending, and descending inputs

Insertion Sort

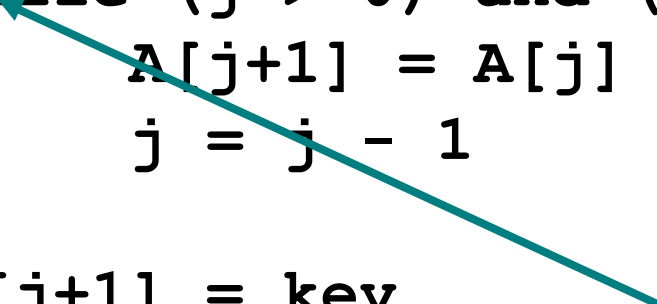
What is the **precondition** for this loop?

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Insertion Sort

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

How many times will this loop execute?



Homework Assignment 1

- Will be released on Wednesday, September 10, 2025
- Due: Wednesday, September 17, 2025 by 11:59 PM

AI Competition Kean Cup

